

SpikingConvNeXt: A Time-to-First-Spike Formulation of ConvNeXt for Sparse Inference

<Author 1>, <Author 2>,

<Affiliation(s)>

<Corresponding author email>

Prepared for submission to .

Abstract

Time-to-first-spike (TTFS) coding is an attractive paradigm for energy-efficient spiking neural networks (SNNs) because each neuron emits at most one spike and information is represented by spike latency, where earlier spikes typically indicate stronger evidence. Recent approximation-free TTFS formulations have shown that deep models can achieve competitive accuracy while operating with sparse spiking activity. In this work, we propose **SpikingConvNeXt**, a TTFS conversion of **ConvNeXt** that (i) removes activation and normalization layers, (ii) replaces pointwise MLP transformations with an analytic spike-time mapping, and (iii) introduces a time-domain residual merge based on an elementwise minimum rather than summation. The minimum residual merge is motivated by TTFS semantics: additive fusion in the time domain can delay spikes and attenuate early evidence, whereas the minimum operator preserves the earliest arriving signal across residual paths. To regulate sparsity, we further introduce learnable per-output delays in the TTFS mapping and optionally constrain pointwise weights to be non-negative to bias neurons toward late or absent spikes. On CIFAR-10, SpikingConvNeXt reaches **95.96%** top-1 accuracy with **33.3%** activation sparsity, and achieves approximately **38%** sparsity with a modest accuracy trade-off (**94.43%**).

Keywords: Spiking neural networks; time-to-first-spike; TTFS; ConvNeXt; sparsity; residual connections; neuromorphic computing

1. Introduction

Spiking neural networks (SNNs) provide an event-driven alternative to conventional artificial neural networks by computing and communicating through discrete spikes. Under time-to-first-spike (TTFS) coding, each neuron emits at most one spike within a fixed time window, and information is represented by spike latency, where earlier spikes typically indicate stronger evidence. This representation is attractive for low-latency inference, since decisions can be formed from early activity, while inactive neurons remain silent and can be represented by late spike times, often clamped to $t_{\max}$.

Despite these advantages, most modern vision backbones are designed for dense real-valued activations and include operations that do not naturally transfer to spike-time representations. ConvNeXt is a representative example: its blocks combine depthwise convolution, a pointwise MLP with value-domain nonlinearities (e.g., GELU), normalization layers, and additive residual connections. When applied directly to spike-time tensors, such operations undermine TTFS semantics. In particular, additive residual fusion in the time domain can shift spike times later, weakening early evidence that is central to TTFS-based computation.

In this work, we present SpikingConvNeXt, a TTFS formulation of ConvNeXt that operates explicitly in the spike-time domain. The proposed approach removes activation and normalization layers and reuses ConvNeXt depthwise and pointwise weights, while replacing the value-domain pointwise MLP pathway with an analytic spike-time mapping adopted from prior approximation-free TTFS formulations. Input images are encoded as spike times using a linear intensity-to-time mapping

over a fixed window. Depthwise convolution is applied directly to spike-time maps for spatial mixing, followed by two pointwise projections implemented via closed-form spike-time updates.

A central architectural modification concerns residual fusion. Standard ConvNeXt merges the residual and transformed paths by addition; however, in TTFS representations, the earliest spike often carries the most salient evidence. We therefore replace residual addition with an elementwise minimum over spike times, which preserves the earliest available evidence across the skip and transformed paths while remaining consistent with TTFS semantics. In addition, we introduce learnable per-output delays in the pointwise TTFS projections, allowing spike timing to be adaptively shifted during training and enabling explicit control over activation sparsity. An optional non-negativity constraint on pointwise weights further biases responses toward late or absent spikes.

Experimental results on CIFAR-10 and CIFAR-100 show that SpikingConvNeXt achieves strong classification performance while maintaining controllable activation sparsity. These findings indicate that modern convolutional backbones can be adapted to TTFS-based spiking networks with minimal structural changes and simple time-domain operators, supporting their potential for efficient deployment on neuromorphic and event-driven hardware.

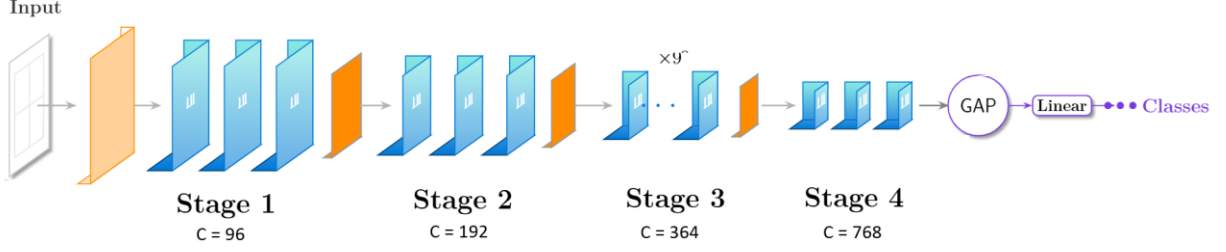


Figure 1 End-to-end architecture of SpikingConvNeXt

2. Proposed Method

2.1. SpikingConvNeXt Overview

We propose SpikingConvNeXt (shown in **Fig. 1**), a time-to-first-spike (TTFS) formulation of ConvNeXt that operates on spike-time representations within a fixed temporal window. The proposed approach preserves the ConvNeXt

macro-architecture while replacing value-domain components with time-domain operators consistent with TTFS semantics. Specifically, input images are encoded as TTFS spike times, the pointwise MLP pathway is replaced with an analytic spike-time transformation, and residual fusion is performed using an elementwise minimum in the spike-time domain. In addition, we introduce learnable delay parameters, and optionally non-negative pointwise weights, to explicitly regulate activation sparsity.

2.2. TTFS Image Encoding and Analytic Spike-Time Mapping

Given an input image $I \in [0,1]^{C \times H \times W}$, we encode each pixel intensity into a spike time within a fixed temporal window $[t_{min}, t_{max}]$ using linear TTFS encoding:

$$t_{in} = t_{min} + (1 - I) * (t_{max} - t_{min})$$

Under this encoding, higher intensity values correspond to earlier spike times. Spike times clamped to t_{max} are treated as no-spike events when computing activation sparsity.

To implement pointwise transformations in the TTFS domain, we compute output spike times directly from input spike times using an analytic mapping. Let $t_j \in R^{B \times C_{in}}$ denote the input spike times, $W \in R^{C_{in} \times C_{out}}$ the weight matrix, and $D \in R^{C_{out}}$ a learnable per-output delay. The mapping is defined as:

$$\theta = (t_{max} - t_{min}) - D, \quad \Delta = t_{in} - t_{min},$$

$$f_{TTFS}(t_{in}) = \Delta * W + \theta + t_{min},$$

2.3. Minimum Residual Merge

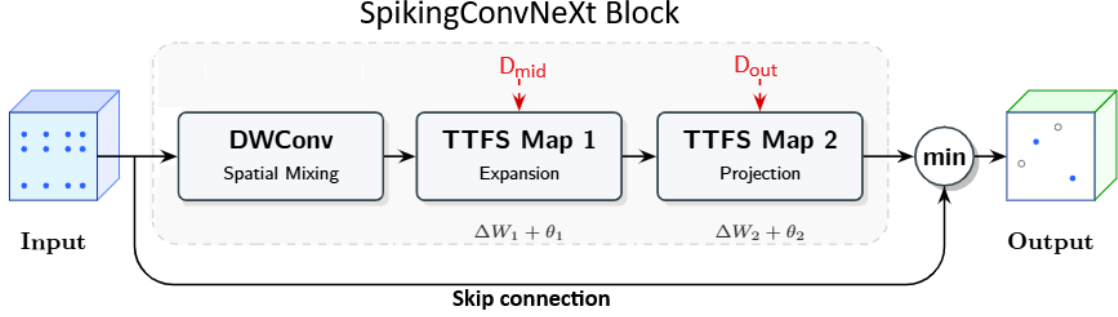
Residual fusion is performed in the spike-time domain via an elementwise minimum:

$$t_{out} = \min(t_{in}, f_{TTFS}(t_{in}))$$

rather than ANN-style addition. This operator preserves the earliest evidence between the shortcut and transformed paths, which aligns with TTFS semantics where earlier spike times indicate stronger evidence. *Figure 2* illustrates the converted block.

2.4. Sparsity Control and Measurement

We incorporate mechanisms to regulate activation sparsity in deep TTFS networks.



Learnable delays. The delay parameters D_{mid} and D_{out} shift spike times during training and can push a subset of activations toward late spike times that saturate at t_{max} , increasing the silent-neuron rate. The delay parameters are constrained to be non-negative and bounded within the time window to ensure valid spike times. The clamping operation enforces the convention that late or absent spikes are represented by t_{max} .

Non-negative pointwise weights. We optionally constrain the pointwise weights to be non-negative (via a ReLU projection). Since $\Delta \geq 0$, enforcing $W \geq 0$, biases the product ΔW toward non-negative shifts, thereby moving output spike times toward later values. Consequently, a larger fraction of outputs exceed t_{max} and are clamped to t_{max} , which we interpret as silent (no-spike) events when computing activation sparsity. This mechanism provides a simple and interpretable way to encourage sparse spiking activity, at the cost of reduced representational flexibility if applied too aggressively.

4. Experimental Results

4.1. Implementation Details

All experiments were implemented using PyTorch and conducted on a single NVIDIA RTX 4060 GPU. Models were trained and evaluated on the CIFAR-10 and CIFAR-100 datasets. Training was performed for a fixed number of epochs using a standard optimization setup and common data augmentation techniques. Unless otherwise stated, reported

results correspond to the model checkpoint achieving the best validation performance.

4.2. Datasets

We evaluate our method on the CIFAR-10 and CIFAR-100 datasets, which are widely used benchmarks for image classification. CIFAR-10 consists of 60,000 color images of size 32×32 , evenly distributed across 10 object categories, with 50,000 training images and 10,000 test images. CIFAR-100 follows the same data split and image resolution but contains 100 fine-grained classes.

4.3. Comparison Experiments and Analysis

We conduct comparison experiments to evaluate the performance of the proposed method against representative spiking and non-spiking neural network baselines on the CIFAR-10 and CIFAR-100 benchmarks. All methods are evaluated using top-1 classification accuracy on the official test sets. For spiking models, we additionally report activation sparsity, defined as the proportion of neurons that remain silent during inference, in order to characterize the efficiency–accuracy trade-off.

Table 1 summarizes the comparison results. The proposed method achieves higher classification accuracy than existing spiking baselines on CIFAR-10 while maintaining a high level of activation sparsity. Compared with conventional ANN baselines, our method demonstrates that spiking networks can achieve competitive or superior accuracy while preserving sparse neural

activity. These results validate the effectiveness of the proposed design in improving classification performance without sacrificing the efficiency advantages of spiking computation. A detailed analysis of individual components and design choices is presented in the ablation study (Section 4.4).

Table 1. Test accuracy and activation sparsity

Method	CIFAR-10		CIFAR-100	
	Acc. (%)	Sparsity (%)	Acc. (%)	Sparsity (%)
Nazari et al. [placeholder ref]	93.60	-	<TBD>	<TBD>
Simonyan (ANN) []	93.56	0 (?)	<TBD>	<TBD>
Stanojevic et al. (TTFS baseline) [2]	93.59	38.00	<TBD>	<TBD>
Ours (SNN) Try 3-B (learnable delays)	94.43	38.01	<TBD>	<TBD>

4.4. Ablation Study

To analyze the contribution of individual components in the proposed method, we conduct ablation experiments on the CIFAR-10 dataset. Unless otherwise stated, all experiments use ConvNeXt-Tiny as the backbone architecture and follow the same training protocol.

We progressively introduce each component of the proposed design and evaluate its impact on classification accuracy and activation sparsity. First, we apply a baseline spiking conversion, in which the pointwise nonlinearity is replaced with a TTFS-based mapping while preserving the original residual merge strategy. Next, we introduce the proposed minimum residual merge, which modifies the residual connection to better preserve early spike information. Finally, we enable learnable delay parameters, allowing the network to adapt spike timing and explicitly regulate sparsity.

Table 3 reports the ablation results. The baseline spiking conversion achieves competitive accuracy with moderate sparsity. Incorporating the minimum residual merge further improves accuracy without increasing sparsity, indicating that the proposed residual formulation better preserves discriminative information in spiking residual blocks. Enabling learnable delays substantially increases activation sparsity, with a moderate reduction in accuracy, revealing a clear trade-off between representational efficiency and classification performance. Overall, the ablation results confirm that each component contributes meaningfully to the final model behavior.

Table 3. Ablation experiment on Cifar10 dataset

Method	Sparsity(%)	Acc. (%)
+Spiking conversion	33.3	95.29
+Minimum residual merge	33.3	95.96
+ Learnable delays	38.5	94.43

Effect of Minimum Residual Merge: In TTFS coding, earlier spike times correspond to larger effective activations. Under conventional additive residual connections, combining spike times through summation can introduce additional latency, which may attenuate early temporal evidence. The proposed minimum residual merge mitigates this effect by preserving the earliest spike between the residual and main branches, providing a plausible explanation for the observed accuracy improvement without increasing sparsity.

Effect of Learnable Delays and Sparsity

Control: Learnable delay parameters enable adaptive modulation of spike timing during training. By shifting a subset of activations toward later spike times that saturate at $t_{\max}$, this mechanism increases the proportion of silent neurons and thus overall activation sparsity. Empirically, we observe that later network

stages are more amenable to sparsification than earlier stages, suggesting that temporal redundancy increases with depth. Further investigation of stage-specific delay strategies is left for future work.

5. Limitations

The proposed method applies depthwise convolution directly to spike-time representations as a practical approximation for spatial mixing. While effective in our experiments, a more principled TTFS-compatible convolution operator may further improve performance. In addition, sparsity-inducing constraints, such as non-negative pointwise weights, introduce a trade-off between activation sparsity and representational capacity, which may depend on the dataset and network stage. Finally, the current evaluation primarily reports accuracy and sparsity; more direct energy and latency measurements would be required to fully characterize deployment efficiency on neuromorphic hardware.

6. Conclusion

In this work, we presented SpikingConvNeXt, a TTFS-based spiking conversion of ConvNeXt that removes explicit activation and normalization layers, replaces pointwise MLP operations with an analytic spike-time mapping, and introduces a minimum residual merge tailored to time-domain representations. The proposed approach achieves strong performance on CIFAR-10, reaching up to 95.96% top-1 accuracy, while enabling controllable activation sparsity through learnable delay parameters. These results indicate that modern convolutional network backbones can be effectively adapted to TTFS-based spiking neural networks using minimal structural modifications and simple temporal operators, supporting their potential for

energy-efficient deployment on neuromorphic and event-driven hardware.

Acknowledgments

<Acknowledgments text (funding, compute grants, collaborators, etc.)>

Data Availability

All datasets used are publicly available (CIFAR-10/100). Exact preprocessing and splits: <provide details / repository link>.

Code Availability

Source code and trained checkpoints will be released at:

github.com/shosseinj/convnextspiking/

References

- [1] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. A ConvNet for the 2020s (ConvNeXt). arXiv:2201.03545, 2022.
- [2] Stanojevic, A., Gehring, W., Woźniak, S., Binas, J., Chicca, E., & Payeur, A. High-performance deep spiking neural networks with 0.3 spikes per neuron. *Nature Communications*, 15:6793, 2024.
- [3] Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report (CIFAR), 2009.
- [4] LeCun, Y., Cortes, C., & Burges, C. J. C. The MNIST database of handwritten digits, 1998.
- [5]

EXTRA INFORMATION

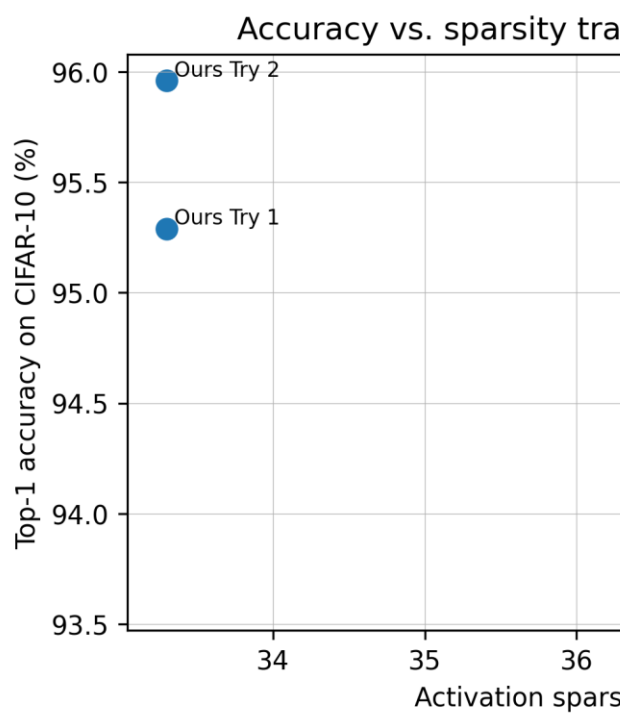


Figure 4. Accuracy vs. sparsity trade-off on CIFAR-10. Minimum-residual variants achieve higher accuracy at comparable sparsity, while learnable delays push sparsity upward with a modest accuracy trade-off.

Figure 5.

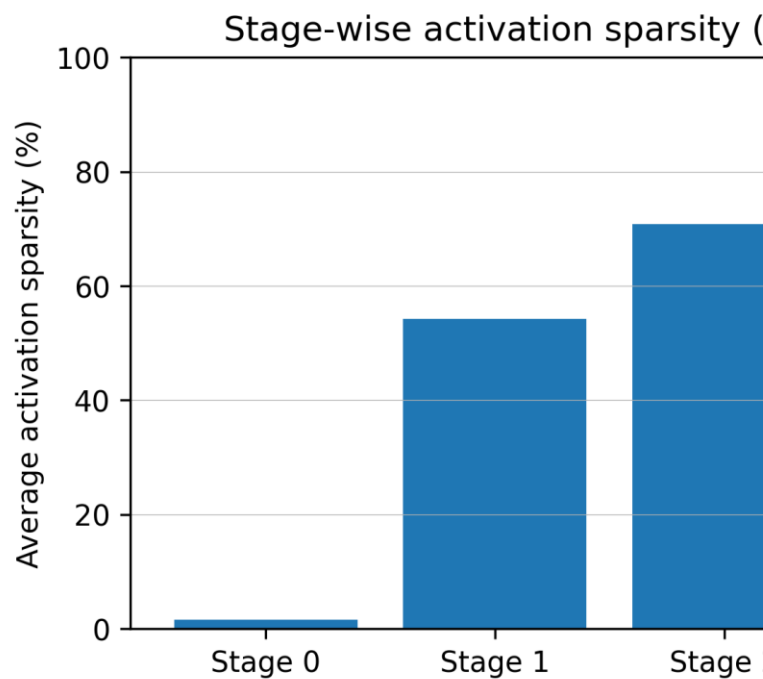


Figure 5. Example stage-wise activation sparsity (Try 3-B snapshot). Early stages exhibit low sparsity compared to later stages, highlighting opportunities to further sparsify early feature extraction.

\$
 \$

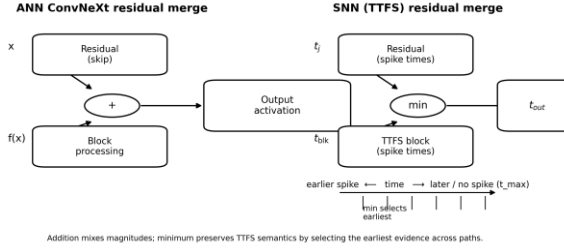


Figure 3. *Residual fusion in ConvNeXt (addition) versus SpikingConvNeXt in the TTFS spike-time domain (minimum). The minimum operator preserves earliest evidence; late or absent spikes are represented by clamping to $t_{\max}$.*

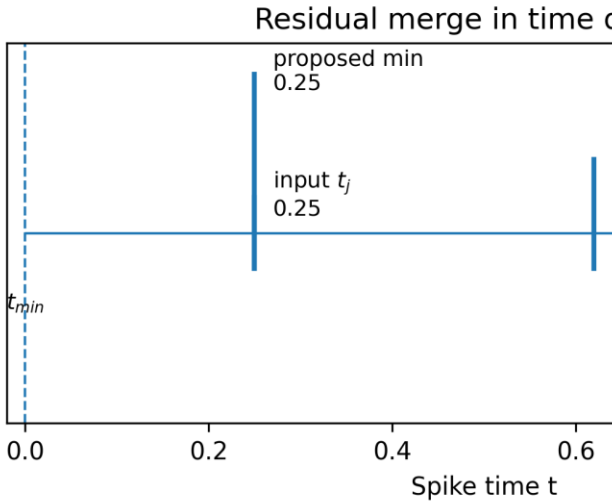


Figure 4. *Illustration of residual merging in the time domain. ANN-style summation delays spike times, while the proposed minimum merge preserves the earliest (most salient) spike.*